

CLARA: Context-aware RAG-LLM Framework for Anomaly Detection in Mobile Device Sensors

Chan Young Koh¹ Kyle DeMedeiros¹ Abdeltawab Hendawi¹

¹ Department of Computer Science and Statistics, University of Rhode Island
 {ckoh04, kyle_demedeiros, hendawi}@uri.edu

Abstract—Mobile devices are equipped with various sensors that continuously gather data about user activity and environmental conditions. Detecting anomalies in this sensor data is crucial for both health monitoring applications and technical quality control. This paper presents a novel framework, namely CLARA, that leverages Retrieval-Augmented Generation (RAG) with Large Language Models (LLMs) to detect anomalies in mobile device sensor data. Using the ExtraSensory dataset, which contains labeled sensor data from smartphones and smartwatches, we demonstrate how RAG enhances LLM-based anomaly detection by retrieving relevant historical patterns and domain knowledge to provide context-rich analysis. Our framework serves dual purposes: (1) providing health and lifestyle insights to end-users through anomaly detection in their daily activities and (2) offering manufacturers a tool for identifying technical sensor malfunctions during quality control processes. The framework delivers rich, contextual explanations of detected anomalies, making the results actionable for end-users and technical teams, addressing key industry challenges in sensor data analysis.

Keywords—Anomaly Detection, Retrieval-Augmented Generation, Large Language Models, Mobile Sensors, Health Monitoring, Quality Control

I. INTRODUCTION

The proliferation of mobile devices equipped with various sensors has created unprecedented opportunities to monitor human activities and environments continuously. Modern smartphones and smartwatches have accelerometers, gyroscopes, magnetometers, and other sensors that generate rich data streams. This sensor data can provide valuable insights into user behavior, health patterns, and device performance [16].

Anomaly detection in sensor data is a critical task with applications ranging from health monitoring to device quality assurance. Traditional anomaly detection methods often struggle with the heterogeneous and noisy nature of mobile sensor data, providing limited meaningful interpretations of detected anomalies. Recent advances in Large Language Models (LLMs) offer new possibilities for advanced anomaly detection systems by leveraging their pattern recognition capabilities and contextual understanding.

However, LLMs alone face limitations when analyzing sensor data, including the potential for hallucinations, a lack of domain-specific knowledge, and an inability to access historical patterns [10], [13]. To address these challenges, we propose integrating Retrieval-Augmented Generation (RAG) with LLM for anomaly detection in mobile sensor data. RAG enhances LLM capabilities by retrieving relevant historical

patterns and domain knowledge from a vector database, providing context-rich analysis that improves detection accuracy and interoperability.

Hence, we present a novel framework that integrates context-aware RAG with LLM to identify and interpret anomalies in mobile device sensor data. Our approach uses the ExtraSensory dataset [17], which contains labeled sensor data from smartphones and smartwatches collected from 60 users during their daily activities. Our framework leverages the unique capabilities of RAG-LLM to directly identify anomalous patterns in sensor data and provide meaningful explanations and contextual interpretations grounded in historical data and domain knowledge.

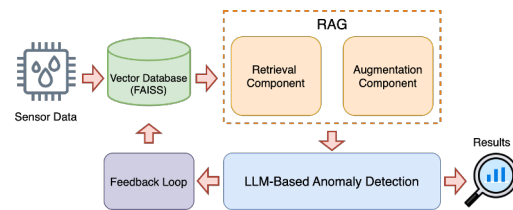


Fig. 1: The architecture of the proposed framework for anomaly detection in mobile device sensors.

The contributions of this paper include (1) a novel framework, CLARA, that integrates RAG with LLMs for mobile sensor data anomaly detection; (2) a methodology for creating and maintaining a vector database of sensor patterns and domain knowledge using Facebook AI Similarity Search (FAISS); (3) an approach for retrieving and augmenting LLM inputs with relevant context for improved analysis; and (4) two application scenarios demonstrating the practical utility of our framework: health/lifestyle monitoring and technical quality control.

The rest of this paper is organized as follows: Section II reviews related work in sensor-based anomaly detection, LLM applications, and RAG. Section III describes the ExtraSensory dataset and our preprocessing and feature extraction methodology. Section IV presents our anomaly detection framework, CLARA. Section V discusses the two application scenarios. Section VI presents the framework's practical implementation and business value. Finally, Section VII concludes the paper and discusses future work.

II. RELATED WORK

A. Mobile Sensor-based Activity Recognition

Activity recognition using mobile sensors has been extensively studied in recent years. Vaizman et al. [16] introduced the ExtraSensory dataset and demonstrated the effectiveness of smartphone and smartwatch sensors for recognizing detailed human contexts in the wild. Their work showed that combining multiple sensors improves recognition accuracy for various daily activities.

Lara and Labrador [9] provided a comprehensive survey of human activity recognition using wearable sensors, discussing various feature extraction methods and classification algorithms. They highlighted the challenges of dealing with heterogeneous sensor data and the importance of contextual information for accurate activity recognition.

B. Anomaly Detection in Sensor Data

Anomaly detection in sensor data has been approached using various techniques. Statistical methods such as Z-score, DBSCAN, and Isolation Forest have been widely used for identifying outliers in sensor data [4]. These methods typically rely on the statistical properties of the data and do not incorporate contextual information.

Deep learning approaches, including autoencoders and recurrent neural networks, have shown promising results for anomaly detection in time-series sensor data [12]. These methods can capture complex temporal patterns but often lack interpretability.

C. Large Language Models in Sensor Data Analysis

The application of Large Language Models (LLMs) to sensor data analysis is a relatively new area of research. Recent work by Brown et al. [3] demonstrated the versatility of LLMs for various tasks beyond natural language processing. Their work suggested that LLMs could be applied to analyze and interpret patterns in diverse data types, including sensor data.

Bommasani et al. [2] discussed the opportunities and challenges of using foundation models, including LLMs, for various applications. They highlighted the potential of these models to transfer knowledge across domains and to provide interpretable insights.

D. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) has emerged as a powerful approach to enhance LLM capabilities by grounding their responses in external knowledge. Lewis et al. [10] introduced RAG to combine neural information retrieval with sequence generation. Their work demonstrated that RAG can improve the factuality and relevance of LLM outputs by retrieving and incorporating relevant information.

In anomaly detection, RAG has been applied to various domains. Russell-Gilbert et al. [15] proposed RAAD-LLM, a framework for adaptive anomaly detection in industrial environments using LLMs integrated with RAG. Their approach showed significant improvements in detection accuracy by leveraging domain-specific knowledge.

Liu et al. [11] developed a RAG-based approach for log anomaly detection, demonstrating how retrieval of relevant log patterns can enhance the detection and interpretation of anomalies in log data. Their work highlighted the importance of effective retrieval strategies for improving anomaly detection performance.

E. Gap in the Literature

Despite the advances in anomaly detection, LLM applications, and RAG, there is a notable gap in integrating these approaches for mobile sensor data analysis. Traditional anomaly detection methods lack flexibility, pure LLM-based approaches suffer from hallucinations and limited domain knowledge, and existing RAG applications have not been fully explored for mobile sensor data. Our work aims to bridge this gap by developing a framework that leverages the strengths of RAG and LLMs for mobile sensor anomaly detection.

III. DATASET AND METHODOLOGY

A. The ExtraSensory Dataset

The ExtraSensory dataset [17] is a large-scale collection of labeled sensor data from smartphones and smartwatches (51 binary contextual labels and 225 features). It contains data from 60 users, each identified with a universally unique identifier (UUID). The dataset includes over 300,000 examples (minutes) of sensor data, each containing measurements from various sensors and context labels self-reported by the user.

The sensors in the dataset include an accelerometer (phone and watch), gyroscope, magnetometer, location services (i.e., GPS), audio features, watch compass, phone state indicators (battery, WiFi, etc.), and additional sensors such as light, pressure, and others. The context labels in the dataset cover a wide range of activities and situations, including physical activities (walking, running, sitting, etc.), locations (home, workplace, school, etc.), device positions (in pocket, on table, in hand, etc.), social contexts (with friends, in a meeting, etc.), and other activities (eating, sleeping, computer work, etc.).

This rich dataset provides an excellent foundation for developing and evaluating our anomaly detection framework, as it captures the complexity and diversity of real-world mobile sensor data.

B. Data Preprocessing

Raw sensor data from mobile devices is often noisy, heterogeneous, and high-dimensional, making it challenging to analyze directly. Our preprocessing pipeline consists of the following steps:

1) Sensor Data Normalization

Different sensors have different scales and units, which can bias anomaly detection. We normalize each sensor's data to have zero mean and unit variance:

$$x_{normalized} = \frac{x - \mu}{\sigma} \quad (1)$$

, where μ is the mean and σ is the standard deviation of the sensor values.

2) Handling Missing Data

The dataset was stored as time-series records, with each row a timestamped multivariate record consisting of many different sensors for a specific volunteer. There were 60 files in total, one for each volunteer. Not all sensors are always available in the ExtraSensory dataset (e.g., from the smartphone not reporting a particular sensor input, a battery dying, etc.). To address this challenge, we employ Generative Adversarial Imputation Networks (GAIN) [18] to handle missing data:

$$\hat{X} = G(X, M, Z) \quad (2)$$

, where $\hat{X}$ is the imputed data, G is the generator network, X is the original data with missing values, M is the mask indicating which values are missing, and Z is a latency matrix, where only missing values are stochastically generated. GAIN preserves the temporal and cross-sensor correlations crucial for accurate anomaly detection, addressing a key challenge in real-world sensor data analysis. The GAIN architecture is illustrated in Figure 2.

The GAIN model is a specialty GAN [18]. Rather than simply taking in a latency vector Z , GAIN also takes in the data input X , as well as a mask matrix M that hides the non-missing elements in X . In Z , the randomly generated elements correspond to only the missing elements in X . The X input has all missing elements set to 0 for initialization purposes. This is represented by X_m in Figure 2.

The resultant output from the generator is a matrix X' that contains all present elements in X , and imputed values created by the generator which replace the missing elements in X . This, along with a hint mechanism reliant on M , is passed as input to the discriminator. The hint mechanism helps the discriminator to predict the mask matrix. This differs from traditional discriminators, which aim to predict whether a given input is entirely real or entirely fake; in GAIN, the discriminator's task is to predict which elements are imputed (fake) and which are not (real), which is equivalent to predicting M . Mean Squared Error is used as the loss function for the generator, while the discriminator uses cross-entropy loss.

Because GAIN aims to learn the mask matrix and output proper imputed data, the authors in [18] employ a combination loss function for the generator. L_G is the loss that applies to the missing components of the input, and L_M is the loss that applies to the observed components of the input. These losses are summed with a modifier α . In addition, as a performance metric, the GAIN implementation utilizes a comparison loss which is similar to L_M but is calculated by using the direct training data rather than the modified input to the generator, as well as using $1 - M$ instead of M .

3) Feature Extraction

We extract comprehensive features from the preprocessed sensor data to capture various aspects of user activities and sensor behaviors. These include statistical features such as mean, variance, skewness, kurtosis (tail heaviness in distributions), and percentiles; temporal features including autocorrelation, trend, seasonality, and change points; frequency-domain features comprising FFT coefficients, spectral entropy, and dominant frequencies; and cross-sensor features that analyze

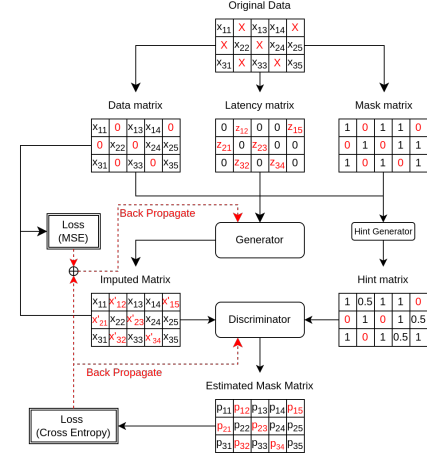


Fig. 2: The GAIN Architecture

correlations between different sensors and mutual information. These features are extracted for each sensor and for different time windows (1 minute, 5 minutes, 15 minutes) to capture both immediate patterns and longer-term trends.

C. Vector Database Creation

A key component of our framework is the vector database that stores embeddings of sensor patterns and domain knowledge. We implement this database using FAISS (Facebook AI Similarity Search) [8], a library specifically designed for efficient similarity search and clustering of dense vectors. We selected FAISS for its exceptional performance in high-dimensional spaces, which is crucial for our sensor embedding representations. We create this database through the following process:

1) Embedding Model

We use a neural network to convert sensor features into vector embeddings:

$$v = f_{\theta}(x) \quad (3)$$

, where v is the vector embedding, f_{θ} is the embedding model with parameters θ , and x is the feature vector.

The embedding model is trained using a contrastive learning approach [7], where similar sensor patterns (e.g., from the same activity or user) are pulled together in the embedding space while dissimilar patterns are pushed apart.

2) Database Population

We populate the FAISS vector database with three types of entries: normal patterns, consisting of embeddings of typical sensor patterns for different users and activities; known anomalies, containing embeddings of labeled anomalous patterns along with their explanations; and domain knowledge, comprising embeddings of expert knowledge about sensor behavior and relationships. Each entry in the database consists of a vector embedding, metadata (e.g., user ID, activity label, timestamp), and a text description that can be retrieved and used in the augmentation process. The resulting embeddings capture the semantic relationships between different sensor patterns, enabling effective retrieval of relevant historical patterns and domain knowledge during anomaly detection.

Algorithm 1 Sensor Embedding Generation with Contrastive Learning

Require: Feature vectors $X = \{x_1, x_2, \dots, x_n\}$, labels $Y = \{y_1, y_2, \dots, y_n\}$, model f_θ
Ensure: Trained embedding model f_θ , embeddings $V = \{v_1, v_2, \dots, v_n\}$

- 1: Initialize model parameters θ randomly
- 2: **for** each epoch **do**
- 3: **for** each mini-batch $B \subset X$ **do**
- 4: $V_B \leftarrow \{f_\theta(x) | x \in B\}$ {Generate embeddings}
- 5: $P \leftarrow \{(i, j) | y_i = y_j, i \neq j, x_i, x_j \in B\}$ {Positive pairs}
- 6: $N \leftarrow \{(i, j) | y_i \neq y_j, x_i, x_j \in B\}$ {Negative pairs}
- 7: $L_{pos} \leftarrow \sum_{(i, j) \in P} \|v_i - v_j\|^2$ {Pull similar patterns together}
- 8: $L_{neg} \leftarrow \sum_{(i, j) \in N} \max(0, m - \|v_i - v_j\|)^2$ {Push dissimilar patterns apart}
- 9: $L \leftarrow L_{pos} + \lambda L_{neg}$ {Combined loss}
- 10: Update θ using gradient descent on L
- 11: **end for**
- 12: **end for**
- 13: $V \leftarrow \{f_\theta(x) | x \in X\}$ {Generate final embeddings}
- 14: **return** f_θ, V

IV. PROPOSED FRAMEWORK

This paper presents CLARA, a Context-aware Language-Augmented Retrieval Anomaly Detection framework for mobile device sensors. Our proposed framework includes detecting and analyzing anomalies in mobile sensor data. By integrating context awareness into an RAG-LLM architecture, we have created a framework that identifies anomalies and provides rich, contextual explanations, making the results immediately actionable for end-users and technical teams.

As shown in Figure 1, our framework consists of five synergistic components that work together to deliver unprecedented capabilities in sensor data analysis. (1) The Vector Database, implemented using FAISS, is a dynamic knowledge repository that stores and organizes patterns and domain expertise. (2) The Retrieval Engine is an intelligent system that finds the most relevant context for current sensor readings. (3) The Augmentation System is a specialized component that structures information for optimal Large Language Model (LLM) processing. (4) The LLM-Based Anomaly Detection component represents a novel approach that leverages language models for pattern recognition, specifically using a small-sized LLM (i.e., Llama 3.2 [1] with 1 billion parameters). We considered the adaptability of our framework to numerous scenarios where systems cannot run large-sized LLMs, such as edge devices or privacy-sensitive applications with limited computational resources. Finally, the (5) Continuous Learning Loop is an adaptive mechanism that improves over time through feedback. This architecture addresses key industry challenges in sensor data analysis, including the need for interpretable results, the ability to handle heterogeneous data, and the capacity to adapt to evolving patterns.

A. Vector Database Component

The vector database serves as the "memory" of our framework, storing embeddings of sensor patterns and domain knowledge. Unlike traditional databases, our FAISS-based

vector database is optimized for semantic similarity search, enabling retrieval of conceptually similar patterns rather than just statistically identical. FAISS provides efficient approximate nearest neighbor search capabilities that scale to millions of vectors, essential for real-world deployments with large amounts of historical sensor data.

The database supports industry-relevant queries with key functionalities. Pattern similarity identifies recurring issues by matching historical patterns to current sensor readings. User-specific patterns retrieve individual user patterns for personalized analysis. Activity-specific patterns focus on patterns tied to specific activities for contextual understanding. Anomaly examples speed up diagnosis by providing known anomaly instances. A feedback loop keeps the database updated with evolving user behaviors and sensor characteristics, which is vital for dynamic, long-term use.

B. Retrieval Component

The retrieval component represents a significant innovation in identifying relevant contexts for anomaly detection. Unlike traditional methods that rely on predefined rules or thresholds, our retrieval system uses FAISS's semantic similarity search to find patterns that are conceptually relevant to the current situation.

The retrieval process employs a multi-query approach that considers different aspects of the current sensor data, ensuring comprehensive coverage of relevant information. This is particularly valuable in industrial settings where anomalies may manifest subtly across multiple sensors.

The retrieved information provides essential context for the LLM analysis, grounding the detection and interpretation in factual data rather than hallucinations. This addresses a key limitation of pure LLM approaches and significantly enhances the reliability of the results for mission-critical applications.

C. Augmentation Component

The augmentation component transforms raw sensor data and retrieved information into a structured format that maximizes the LLM's ability to detect and interpret anomalies. This component addresses the fundamental challenge of applying language models to structured sensor data.

Algorithm 2 Sensor Data Augmentation for LLM Processing

Require: Sensor data S , retrieved context R
Ensure: Augmented prompt P

- 1: $T_S \leftarrow \text{SensorToText}(S)$ {Sensor data to descriptive text}
- 2: $T_R \leftarrow \text{ContextToText}(R)$ {Structure retrieved context}
- 3: $T_G \leftarrow \text{GenerateGuidance}(S, R)$ {Guidance for LLM}
- 4: $T_P \leftarrow \text{PatternHighlighting}(S, R)$ {Highlight key patterns}
- 5: $P \leftarrow \text{CombineComponents}(T_S, T_R, T_G, T_P)$
- 6: **return** P

As shown in Algorithm 2, our augmentation approach converts numerical sensor readings into descriptive text, highlighting key patterns and structures retrieved information to emphasize the most relevant aspects. It explicitly guides the LLM on what to focus on and balances detail and conciseness to work within token limits. This structured approach ensures

that the LLM's analysis focuses on the most informative patterns and relationships, significantly improving detection accuracy and explanation quality in real-world deployments.

D. LLM-Based Anomaly Detection

Our framework introduces a context-aware anomaly detection approach that leverages LLMs' unique capabilities. Instead of relying on statistical thresholds or predefined rules, we employ a multi-faceted detection system that understands the semantic meaning of patterns in context.

1) Semantic Pattern Matching

Our framework identifies anomalies by comparing current sensor patterns with semantically similar historical patterns:

$$S(p_{\text{current}}, P_{\text{historical}}) = \text{sim}(v_{\text{current}}, V_{\text{historical}}) \quad (4)$$

, where the function, $S(p_{\text{current}}, P_{\text{historical}})$, denotes the similarity score, p_{current} represents the current sensor pattern being analyzed for potential anomalies, and set $P_{\text{historical}}$ refers to the collection of historical sensor patterns stored in the database. The function sim is the similarity measure used to compare embeddings, where v_{current} and $V_{\text{historical}}$ represent the vector embeddings of the current pattern and historical patterns, respectively.

This approach enables the detection of subtle anomalies by comparing current patterns with semantically similar historical ones rather than relying on fixed statistical thresholds.

2) Contextual Deviation Analysis

Rather than using fixed thresholds, the framework analyzes deviations in the context of user-specific behavioral patterns, activity-specific normal variations, environmental factors, and temporal context:

$$D(p, c) = \text{LLM}_{\text{deviation}}(p, R_c) \quad (5)$$

The function $D(p, c)$ denotes the contextual deviation score quantifying how much a pattern deviates from what's expected in a specific context. The variable p represents the sensor pattern being analyzed for potential anomalies. The context c encompasses user-specific, activity-specific, environmental, and temporal factors influencing what should be considered normal. The function $\text{LLM}_{\text{deviation}}$ represents the specialized LLM component that performs the deviation analysis. The variable R_c refers to the retrieved context-specific information from the vector database that provides relevant historical context.

We can reduce false positives by analyzing deviations in the context of user-specific behavioral patterns and other contextual factors while maintaining sensitivity to significant deviations.

3) Multi-modal Pattern Recognition

The LLM component recognizes complex patterns across multiple sensors and modalities:

$$M(S_1, S_2, \dots, S_n) = \text{LLM}_{\text{multi}}(S_1, S_2, \dots, S_n, R_{\text{cross}}) \quad (6)$$

, where $M(S_1, S_2, \dots, S_n)$ denotes the multi-modal pattern recognition result that identifies relationships across different sensor types, $S_1, S_2, \dots, S_n$ represent readings from different sensors that are being analyzed together, $\text{LLM}_{\text{multi}}$ refers to the

Algorithm 3 Contextual Deviation Analysis

Require: Sensor pattern p , context c , retrieved patterns R_c , LLM $\mathcal{L}$
Ensure: Contextual deviation score a_{ctx} , explanation e_{ctx}
1: $T_p \leftarrow \text{PatternToText}(p)$ {Pattern to text}
2: $T_c \leftarrow \text{ContextToText}(c)$ {Context to text}
3: $T_R \leftarrow \text{RetrievedToText}(R_c)$ {Retrieved patterns to text}
4: $P_{ctx} \leftarrow \text{ConstructPrompt}(T_p, T_c, T_R)$ {Construct prompt}
5: $(a_{ctx}, e_{ctx}) \leftarrow \mathcal{L}(P_{ctx})$ {Get LLM analysis}
6: $a_{ctx} \leftarrow \text{NormalizeScore}(a_{ctx})$ {Normalize score to [0,1]}
7: **return** (a_{ctx}, e_{ctx})

specialized LLM component that performs multi-modal analysis across sensor types, and R_{cross} represents the retrieved cross-sensor contextual information that provides historical context about normal relationships between sensors. This capability enables the detection of anomalies that manifest as unusual relationships between different sensors, addressing a key challenge in complex systems where issues often span multiple sensors.

4) Explanation-Driven Detection

The framework integrates explanation generation into the detection process:

$$\{a, c, e\} = \text{LLM}_{\text{explain}}(p, R) \quad (7)$$

The output tuple $\{a, c, e\}$ contains three essential components: a is the anomaly flag (a boolean indicating the presence of an anomaly), c is the confidence score (a numeric value indicating detection confidence), and e is the explanation (a textual description of the anomaly). The function $\text{LLM}_{\text{explain}}$ represents the specialized LLM component that generates explanations as part of the detection process.

Algorithm 4 Explanation-Driven Anomaly Detection

Require: Sensor pattern p , retrieved context R , LLM $\mathcal{L}$, coherence threshold γ
Ensure: Anomaly flag a , confidence score c , explanation e
1: $T_p \leftarrow \text{PatternToText}(p)$ {Convert pattern to text}
2: $T_R \leftarrow \text{ContextToText}(R)$ {Convert context to text}
3: $P_{exp} \leftarrow \text{ConstructExplainPrompt}(T_p, T_R)$ {Construct prompt}
4: $(a', c', e) \leftarrow \mathcal{L}(P_{exp})$ {Get initial LLM analysis}
5: $\text{coherence} \leftarrow \text{EvaluateCoherence}(e, p, R)$
6: **if** $\text{coherence} < \gamma$ **then**
7: $a \leftarrow \text{False}$ {Reject detection if explanation incoherent}
8: $c \leftarrow c' \cdot \text{coherence}$ {Adjust confidence by coherence}
9: **else**
10: $a \leftarrow a'$ {Accept LLM detection decision}
11: $c \leftarrow c'$ {Use LLM confidence score}
12: **end if**
13: **return** (a, c, e)

We create Algorithm 4 to derive 7, which allows our framework to reduce spurious detections by using the ability to generate a coherent explanation as part of the criteria for identifying true anomalies. It provides immediate context for understanding detected issues.

E. Prompt Engineering for Non-Trainable LLMs

Our framework emphasizes the practical application of prompt engineering techniques for non-retrainable LLMs. This

addresses a key practical challenge in industrial deployments, where retraining large models for specific domains is often prohibitively expensive or technically infeasible [6].

The challenge stems from the inherent limitations of general-purpose LLMs when processing structured sensor data. *These models are primarily trained on natural language and do not understand the temporal, spatial, and cross-sensor relationships crucial for anomaly detection.* Through systematic experimentation, we developed a prompt engineering methodology that addresses these limitations:

$$P(F, A, C) = T_{\text{intro}} \oplus T_{\text{data}}(F) \oplus T_{\text{anomaly}}(A) \oplus T_{\text{context}}(C) \oplus T_{\text{directive}} \quad (8)$$

, where P is the final prompt, F represents the feature data, A represents the anomaly scores, C represents contextual information, T_{intro} is the introductory text that frames the task, T_{data} is a function that converts feature data to text, T_{anomaly} converts anomaly scores to text, T_{context} converts contextual information to text, $T_{\text{directive}}$ provides specific instructions to the LLM, and $\oplus$ represents text concatenation.

F. Feedback Loop

The feedback loop represents a key innovation enabling continuous framework improvement. Unlike static anomaly detection systems, our framework learns from each analysis and user feedback:

$$DB' = DB \cup \{(v_{\text{new}}, d_{\text{new}}, m_{\text{new}})\} \quad (9)$$

, where DB' denotes the updated vector database after incorporating new information, and DB represents the current vector database before the update. v_{new} refers to the vector embedding of the newly identified pattern, and d_{new} contains the textual information about the new pattern. In contrast, m_{new} represents the metadata associated with the new pattern, such as timestamps and source information.

This continuous learning approach ensures that the framework adapts to evolving sensor behaviors and user activities, thereby maintaining detection accuracy over time, a critical requirement for long-term industrial deployments.

G. Implementation Architecture

Algorithm 5 describes the implementation of CLARA. CLARA is designed with a focus on flexibility, scalability, and ease of integration with existing systems. The modular architecture allows components to be deployed across different environments based on specific requirements. The Vector Database is implemented using Facebook AI Similarity Search (FAISS) [8], which provides efficient similarity search capabilities even with millions of vectors. We specifically utilize FAISS's IndexFlatL2 for exact nearest neighbor search during development and IndexIVFPQ for approximate nearest neighbor search in production, balancing accuracy and performance [8]. The Embedding Model is a specialized neural network that maps sensor features to a high-dimensional embedding space. The Retrieval Engine is a multi-stage system that finds, ranks and filters relevant information using FAISS's search capabilities. The Augmentation System employs a template-based approach that structures information for optimal LLM

Algorithm 5 Context-aware Language-Augmented Retrieval Anomaly Detector

Require: Sensor data S , FAISS index I , LLM $\mathcal{L}$, thresholds θ, γ
Ensure: Anomaly detection result $\{a, c, e, t\}$

```

1:  $R \leftarrow \text{MultiQueryRetrieval}(S, I)$ 
2:  $P \leftarrow \text{SensorDataAugmentation}(S, R)$ 
3:  $(a_{\text{sem}}, M) \leftarrow \text{SemanticPatternMatching}(S, R, \theta)$ 
4:  $(a_{\text{ctx}}, e_{\text{ctx}}) \leftarrow \text{ContextualDeviationAnalysis}(S, R, \mathcal{L})$ 
5:  $(a_{\text{multi}}, e_{\text{multi}}) \leftarrow \text{MultiModalPatternRecognition}(S, R, \mathcal{L})$ 
6:  $(a_{\text{exp}}, c_{\text{exp}}, e_{\text{exp}}) \leftarrow \text{ExplanationDrivenDetection}(S, R, \mathcal{L}, \gamma)$ 
7:  $a \leftarrow \text{CombineDetections}(a_{\text{sem}}, a_{\text{ctx}}, a_{\text{multi}}, a_{\text{exp}})$ 
8:  $c \leftarrow \text{CombineConfidences}(a_{\text{sem}}, a_{\text{ctx}}, a_{\text{multi}}, c_{\text{exp}})$ 
9:  $e \leftarrow \text{EnhanceExplanation}(e_{\text{ctx}}, e_{\text{multi}}, e_{\text{exp}}, M)$ 
10:  $t \leftarrow \text{DetermineAnomalyType}(S, R, a, e)$ 
11: return  $\{a, c, e, t\}$ 

```

processing. The LLM Integration provides flexible integration with various open-source LLMs through Ollama [14], with structured output parsing. The implementation supports multiple deployment options, including local deployment for privacy-sensitive applications, edge-cloud hybrid for balanced performance and privacy, and complete cloud deployment for maximum scalability.

V. APPLICATION SCENARIOS

CLARA is designed to serve two primary application scenarios: health and lifestyle monitoring, and technical quality control. This section describes how it can be applied in each scenario.

A. Health and Lifestyle Monitoring

1) Scenario Overview

In this application scenario, our framework is utilized by a health and lifestyle tracking application to provide users with insights into their daily activities and potential health concerns. The application continuously collects sensor data from the user's smartphone and smartwatch, processes it through our framework, and presents the results to the user in a comprehensive format.

2) Key Features

The framework provides several key features for health and lifestyle monitoring. It performs activity pattern analysis by identifying the user's regular activity patterns and detecting deviations that might indicate changes in health or lifestyle. It employs contextual anomaly detection by considering the context (such as location and time) to distinguish between normal variations and genuine anomalies. The framework generates health insights through the LLM component, which produces natural language insights about detected anomalies, explaining their potential significance for the user's health. Additionally, it conducts trend analysis by tracking changes in sensor patterns over time, identifying gradual shifts that might not be apparent in short-term analysis.

3) Example Use Cases

The framework can be applied to detect changes in sleep patterns that might indicate stress or health issues, identify

unusual physical activity levels that deviate from the user's normal routine, recognize potential signs of sedentary behavior or excessive physical strain, and monitor changes in daily routines that might correlate with mood or mental health.

B. Technical Quality Control

1) Scenario Overview

In this application scenario, our framework is used by device manufacturers for quality control and debugging of mobile device sensors. The framework analyzes sensor data collected during testing phases to identify potential hardware or software issues that affect the user experience.

2) Key Features

The framework provides several key features for technical quality control. It performs sensor performance analysis by evaluating the performance of individual sensors against expected specifications. It conducts cross-sensor correlation analysis by analyzing sensor relationships to identify inconsistencies that might indicate calibration issues. The framework utilizes anomaly classification through the LLM component, categorizing detected anomalies as hardware defects, software bugs, or calibration issues. Additionally, it performs root cause analysis by generating hypotheses about the potential causes of detected anomalies, helping engineers focus their debugging efforts.

3) Example Use Cases

The framework can be applied to detect accelerometer drift or calibration issues, identify gyroscope instability under specific conditions, recognize GPS signal quality problems in certain environments, and monitor battery performance and its impact on sensor readings.

4) Integration with Manufacturing Processes

The framework can be integrated into the manufacturing and quality assurance processes through automated testing of devices on the production line, continuous monitoring of returned devices to identify common issues, analysis of field data to detect emerging problems, and a feedback loop for improving sensor design and calibration.

C. Comparative Analysis of the Two Scenarios

While both application scenarios use the same underlying framework, they differ in several key aspects. These differences influence how the framework is configured and deployed in each scenario, but the core methodology remains the same, demonstrating the flexibility and adaptability of our approach.

VI. RESULTS AND EVALUATION

A. GAIN imputation

We pre-trained GAIN on the dataset with a learning rate of 0.0001 and 100% data to intentionally overfit for precise imputation before input into the language model (LM). Separately, we trained GAIN with an 80-20 train-test split to report training and testing losses. The hyperparameters used in both cases include 3 MLP layers, ReLU and Sigmoid activations, Adam optimizer, batch size 128, missing rate 20, hint rate 90, and $\alpha = 10$. Since imputation normalizes the data, we denormalized the imputed values by scaling them with the

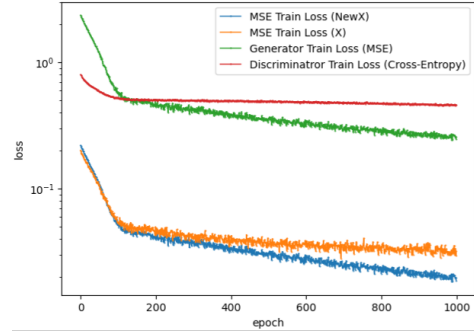


Fig. 3: GAIN Training Loss Values for ExtraSensory Dataset.

value of the closest non-missing neighbor in the original data, matched by host ID, to ensure correct units and scale. This provides complete data for Retrieval-Augmented Generation (RAG), which is essential for Language Model (LM) accuracy, except when all data for a feature of a host ID is missing.

We show the training loss values for the ExtraSensory Dataset in Figure 3. In this figure, we use a log scale (y-axis: loss, x-axis: epoch) to illustrate how closely the loss values are clustered together. The MSE train loss using the modified (new X) and original (X) data with a mask is very close. This indicates that the generator is learning the data as it trains. The results are as expected; the loss decreases as training continues. GAIN quickly learns the data structure in a couple of iterations.

B. Anomaly Detection Performance

TABLE I: Anomaly Detection Performance

Method	Precision	Recall	F1-score	AUC-ROC
LLM (llama3.2:1b)	0.81	0.77	0.79	0.85
Traditional RAG	0.85	0.80	0.82	0.87
Traditional RAG-LLM	0.87	0.83	0.85	0.91
Ours: CLARA	0.93	0.89	0.91	0.95

Our results showed that CLARA outperformed all baseline methods in terms of anomaly detection performance, as shown in Table I. The Traditional RAG and RAG-LLM are vanilla implementations of RAG and RAG-LLM, utilizing the LangChain library [5]. The superior performance of our framework can be attributed to several key factors. First, semantic pattern matching enables the detection of subtle anomalies based on semantic similarity rather than relying solely on statistical thresholds. This is complemented by contextual deviation analysis, which considers the full context of sensor readings, thereby reducing false positives from normal variations. Additionally, multi-modal pattern recognition identifies complex anomalies across multiple sensors, ensuring a more comprehensive detection approach. Finally, explanation-driven detection utilizes the ability to generate coherent explanations as part of its detection criteria, thereby adding an extra layer of validation to the anomaly detection process.

C. Performance on Different Anomaly Types

We further analyzed performance across different types of anomalies. As shown in Table II, CLARA showed robust performance on contextual and collective anomalies, which traditional methods typically find challenging. This demonstrates the value of our semantic pattern matching and contextual deviation analysis approaches.

TABLE II: F1-scores for Different Anomaly Types

Method	Point	Contextual	Collective	Seasonal
LLM (llama3.2:1b)	0.82	0.76	0.74	0.71
Traditional RAG	0.84	0.81	0.79	0.75
Traditional RAG-LLM	0.86	0.83	0.82	0.78
CLARA	0.92	0.91	0.89	0.87

D. Sensor Importance Analysis

CLARA also provides insights into which sensors are most important for anomaly detection in different application scenarios. Table III shows the importance scores for different sensors across various use cases.

TABLE III: Sensor Importance Scores by Use Case

Sensor	Physical Activity	Sleep Patterns	Stress Levels	Dietary Habits
Accelerometer	0.85	0.85	0.82	0.85
Gyroscope	0.76	0.30	0.45	0.25
Watch Accelerometer	0.82	0.94	0.92	0.85
Location (GPS)	0.65	0.25	0.65	0.90
Audio	0.40	0.72	0.55	0.78
Phone State	0.35	0.45	0.88	0.72

These importance scores highlight the value of different sensors for specific use cases. For example, watch accelerometer data is particularly important for monitoring sleep patterns and stress levels, while location data is crucial for monitoring dietary habits.

VII. CONCLUSION

In this paper, we introduce CLARA, a novel framework that uses context-aware RAG-LLM as the primary mechanism for anomaly detection in mobile device sensor data. Unlike traditional approaches that rely on statistical methods, our framework leverages LLMs' pattern recognition and contextual understanding capabilities, enhanced by retrieving relevant historical patterns and contexts from a vector database. Our framework serves dual purposes: providing health and lifestyle insights to end-users and offering manufacturers a tool for technical quality control. The LLM component enables the generation of contextual, specific, and actionable explanations of detected anomalies, addressing a key limitation of traditional anomaly detection methods.

Several directions for future work emerge from this research. We will develop methods to ensure data privacy when using LLMs for sensor data analysis. We aim to extend the framework to incorporate other data modalities, such as images and audio. We will improve the framework to adapt to individual users' patterns and preferences over time. Finally, we

plan to improve the framework's efficiency to enable real-time anomaly detection and interpretation. These future directions will further improve the capabilities and applicability of our framework in various domains.

REFERENCES

- [1] Meta AI. Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models. Meta AI Blog (AI at Meta), 2024.
- [2] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.
- [3] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.
- [4] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.
- [5] Harrison Chase and LangChain contributors. Langchain: Building applications with llms. <https://github.com/langchain-ai/langchain>, 2022. Version accessed: April 2025.
- [6] Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. Unleashing the potential of prompt engineering in large language models: a comprehensive review. *arXiv preprint arXiv:2310.14735*, 2023.
- [7] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20, pages 1597–1607, 2020.
- [8] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. 2024.
- [9] Oscar D Lara and Miguel A Labrador. A survey on human activity recognition using wearable sensors. *IEEE Communications Surveys & Tutorials*, 15(3):1192–1209, 2013.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [11] Yilun Liu, Shimin Tao, Weibin Meng, Jingyu Wang, Wenbing Ma, Yanqing Zhao, Yuhang Chen, Hao Yang, Yanfei Jiang, and Xun Chen. Interpretable online log analysis using large language models with prompt strategies. In *Proceedings of the International Conference on Program Comprehension (ICPC)*, 2024. First published on arXiv, Aug 15, 2023, arXiv:2308.07610.
- [12] Pankaj Malhotra, Lovekesh Vig, Gautam Shroff, and Puneet Agarwal. Long short term memory networks for anomaly detection in time series. In *Proceedings of the European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*, pages 89–94, 2015.
- [13] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. Augmented language models: A survey. *arXiv preprint arXiv:2302.07842*, 2023.
- [14] Ollama Development Team. Ollama, 2025. Version 0.6.2. Available from <https://github.com/ollama/ollama>.
- [15] Alicia Russell-Gilbert, Sudip Mittal, Shahram Rahimi, Maria Seale, Joseph Jabour, Thomas Arnold, and Joshua Church. Raad-llm: Adaptive anomaly detection using llms and rag integration. *arXiv preprint arXiv:2503.02800*, 2025.
- [16] Yonatan Vaizman, Katherine Ellis, and Gert Lanckriet. Recognizing detailed human context in the wild from smartphones and smartwatches. *IEEE Pervasive Computing*, 16(4):62–74, 2017.
- [17] Yonatan Vaizman, Katherine Ellis, Gert Lanckriet, and Nadir Weibel. Extrasensory app: Data collection in-the-wild with rich user interface to self-report behavior. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, pages 1–12. ACM, 2018.
- [18] Jinsung Yoon, James Jordon, and Mihaela van der Schaar. Gain: Missing data imputation using generative adversarial nets, 2018.